chapter

12

Parallel Processing and Performance

Chapter Objectives

In this chapter you will learn about:

- Multithreading
- Vector processing in general-purpose and graphics processors
- Shared-memory multiprocessors
- Cache coherence for shared data
- Message passing in distributed-memory systems
- Parallel programming
- Mathematical modeling of performance

In Chapter 6, instruction pipelining and superscalar operation are described as processor design techniques aimed at increasing the rate of instruction execution, which therefore reduces execution time. In Chapter 8, caches are presented as a way to reduce the average latency in accessing instructions and data.

This chapter describes three additional techniques for improving performance, namely multithreading, vector processing, and multiprocessing. These techniques are implemented in the multicore processor chips that are used in general-purpose computers. They increase performance by improving the utilization of processing resources and by performing more operations in parallel.

12.1 HARDWARE MULTITHREADING

Operating system (OS) software enables multitasking of different programs in the same processor by performing context switches among programs. As explained in Section 4.9, a program, together with any information that describes its current state of execution, is regarded by the OS as an entity called a *process*. Information about the memory and other resources allocated by the OS is maintained with each process. Processes may be associated with applications such as Web-browsing, word-processing, and music-playing programs that a user has opened in a computer.

Each process has a corresponding thread, which is an independent path of execution within a program. More precisely, the term *thread* is used to refer to a thread of control whose state consists of the contents of the program counter and other processor registers. As we will discuss in Section 12.6, it is possible for multiple threads to execute portions of one program and run in parallel as if they correspond to separate programs. Two or more threads can be running on different processors, executing either the same part of a program on different data, or executing different parts of a program. Threads for different programs can also execute on different processors. All threads that are part of a single program run in the same address space and are associated with the same process.

In this section, we focus on multitasking where two or more programs run on the same processor and each program has a single thread. Section 4.9 describes the technique of time slicing, where the OS selects a process among those that are not presently blocked and allows this process to run for a short period of time. Only the thread corresponding to the selected process is active during the time slice. Context switching at the end of the time slice causes the OS to select a different process, whose corresponding thread becomes active during the next time slice. A timer interrupt invokes an interrupt-service routine in the OS to switch from one process to another.

To deal with multiple threads efficiently, a processor is implemented with several identical sets of registers, including multiple program counters. Each set of registers can be dedicated to a different thread. Thus, no time is wasted during a context switch to save and restore register contents. The processor is said to be using a technique called *hardware multithreading*.

With multiple sets of registers, context switching is simple and fast. All that is necessary is to change a hardware pointer in the processor to use a different set of registers to fetch and execute subsequent instructions. Switching to a different thread can be completed within

one clock cycle. The state of the previously active thread is preserved in its own set of registers.

Switching to a different thread may be triggered at any time by the occurrence of a specific event, rather than at the end of a fixed time interval. For example, a cache miss may occur when a Load or Store instruction is being executed for the active thread. Instead of stalling while the slower main memory is accessed to service the cache miss, a processor can quickly switch to a different thread and continue to fetch and execute other instructions. This is called *coarse-grained* multithreading because many instructions may be executed for one thread before an event such as a cache miss causes a switch to another thread.

An alternative to switching between threads on specific events is to switch after every instruction is fetched. This is called *fine-grained* or *interleaved* multithreading. The intent is to increase the processor throughput. Each new instruction is independent of its predecessors from other threads. This should reduce the occurrence of stalls due to data dependencies. Thus, throughput may be increased by interleaving instructions from many threads, but it takes longer for a given thread to complete all of its instructions. A form of interleaved multithreading with only two threads is used in processors that implement the Intel IA-32 architecture described in Appendix E.

12.2 VECTOR (SIMD) PROCESSING

Many computationally demanding applications involve programs that use loops to perform operations on vectors of data, where a *vector* is an array of elements such as integers or floating-point numbers. When a processor executes the instructions in such a loop, the operations are performed one at a time on individual vector elements. As a result, many instructions need to be executed to process all vector elements.

A processor can be enhanced with multiple ALUs. In such a processor, it is possible to operate on multiple data elements in parallel using a single instruction. Such instructions are called *single-instruction multiple-data* (SIMD) instructions. They are also called *vector instructions*. These instructions can only be used when the operations performed in parallel are independent. This is known as *data parallelism*.

The data for vector instructions are held in *vector registers*, each of which can hold several data elements. The number of elements, L, in each vector register is called the *vector length*. It determines the number of operations that can be performed in parallel on multiple ALUs. If vector instructions are provided for different sizes of data elements using the same vector registers, L may vary. For example, the Intel IA-32 architecture has 128-bit vector registers that are used by instructions for vector lengths ranging from $L = 2$ up to $L = 16$, corresponding to integer data elements with sizes ranging from 64 bits down to 8 bits.

Some typical examples of vector instructions are given below to illustrate how vector registers are used. We assume that the OP-code mnemonic includes a suffix S which specifies the size of each data element. This determines the number of elements, L, in a vector. For instructions that access the memory, the contents of a conventional register are used in the calculation of the effective address.

The vector instruction

VectorAdd.S V*i*, V*j*, V*k*

computes *L* sums using the elements in vector registers V*j* and V*k*, and places the resulting sums in vector register V*i*. Similar instructions are used to perform other arithmetic operations.

Special instructions are needed to transfer multiple data elements between a vector register and the memory. The instruction

VectorLoad.S V*i*, X(R*j*)

causes *L* consecutive elements beginning at memory location X + [R*j*] to be loaded into vector register V*i*. Similarly, the instruction

VectorStore.S V*i*, X(R*j*)

causes the contents of vector register V*i* to be stored as *L* consecutive elements in the memory.

Vectorization

In a source program written in a high-level language, loops that operate on arrays of integers or floating-point numbers are *vectorizable* if the operations performed in each pass are independent of the other passes. Using vector instructions reduces the number of instructions that need to be executed and enables the operations to be performed in parallel on multiple ALUs. A *vectorizing compiler* can recognize such loops, if they are not too complex, and generate vector instructions.

Vectorization of a loop is illustrated with the simple example of C-language code shown in Figure 12.1*a*. Assume that the starting locations in memory for arrays A, B, and C are in registers R2, R3, and R4. Using conventional assembly-language instructions, the compiler may generate the loop shown in Figure 12.1*b*. The loop body has nine instructions, hence a total of 9*N* instructions are executed for *N* passes through the loop.

To vectorize the loop, the compiler must recognize that the computation in each pass through the loop is independent of the other passes, and that the same operation can be performed simultaneously on multiple elements. Assume for simplicity that the number of passes, *N*, is evenly divisible by the vector length, *L*. The Load, Add, and Store instructions at the beginning of the loop are replaced by corresponding vector instructions that operate on *L* elements at a time. Consequently, the vectorized loop requires only N/L passes to process all of the data in the arrays. With *L* elements processed in each pass through the loop, the address pointers in registers R2, R3, and R4 are incremented by 4*L*, and the count in register R5 is decremented by *L*. The vectorized loop is shown in Figure 12.1*c*. We assume that the assembler calculates the value of 4*L* for the expression given as the immediate operand in the Add instructions. There are still nine instructions in the loop body, but because the number of passes is now N/L, the total number of instructions that need to be executed is only $9N/L$.

Vectorizable loops exist in programs for applications such as computer graphics and digital signal processing. Such loops have many independent computations that can be performed simultaneously on several data elements. If a large portion of the execution time

```
for (i = 0; i < N; i++)
   A[i] = B[i] + C[i];
```

(a) A C-language loop to add vector elements

```
        Move            R5, #N          R5 is the loop counter.
LOOP:   Load            R6, (R3)        R3 points to an element in array B.
        Load            R7, (R4)        R4 points to an element in array C.
        Add             R6, R6, R7      Add a pair of elements from the arrays.
        Store           R6, (R2)        R2 points to an element in array A.
        Add             R2, R2, #4      Increment the three array pointers.
        Add             R3, R3, #4
        Add             R4, R4, #4
        Subtract        R5, R5, #1      Decrement the loop counter.
        Branch_if_[R5]> 0 LOOP          Repeat the loop if not finished.
```

(b) Assembly-language instructions for the loop

```
        Move            R5, #N          R5 counts the number of elements to process.
LOOP:   VectorLoad.S    V0, (R3)        Load L elements from array B.
        VectorLoad.S    V1, (R4)        Load L elements from array C.
        VectorAdd.S     V0, V0, V1      Add L pairs of elements from the arrays.
        VectorStore.S   V0, (R2)        Store L elements to array A.
        Add             R2, R2, #4*L    Increment the array pointers by L words.
        Add             R3, R3, #4*L
        Add             R4, R4, #4*L
        Subtract        R5, R5, #L      Decrement the loop counter by L.
        Branch_if_[R5]> 0 LOOP          Repeat the loop if not finished.
```

(c) Vectorized form of the loop

Figure 12.1 Example of loop vectorization.

for an application is spent executing loops of this type, then vectorization of these loops can significantly reduce the total execution time. The extent of the performance improvement is limited by the vector length L, which determines the number of ALUs that can operate in parallel. To obtain higher performance, support for vector (SIMD) processing can be implemented in another way, as the next section describes.

12.2.1 GRAPHICS PROCESSING UNITS (GPUs)

The increasing demands of processing for computer graphics has led to the development of specialized chips called *graphics processing units* (GPUs). The primary purpose of GPUs is to accelerate the large number of floating-point calculations needed in high-resolution three-dimensional graphics, such as in video games. Since the operations involved in these calculations are often independent, a large GPU chip contains hundreds of simple cores with floating-point ALUs to perform them in parallel.

A GPU chip and a dedicated memory for it are included on a video card. Such a card is plugged into an expansion slot of a host computer using an interconnection standard such as the PCIe standard discussed in Chapter 7. A small program is written for the processing cores in the GPU chip. A large number of cores execute this program in parallel. The cores execute the same instructions, but operate on different data elements. A separate controlling program runs in the general-purpose processor of the host computer and invokes the GPU program when necessary. Before initiating the GPU computation, the program in the host computer must first transfer the data needed by the GPU program from the main memory into the dedicated GPU memory. After the computation is completed, the resulting output data in the dedicated memory are transferred back to the main memory.

The processing cores in a GPU chip have a specialized instruction set and hardware architecture, which are different from those used in a general-purpose processor. An example is the *Compute Unified Device Architecture* (CUDA) that NVIDIA Corporation uses for the cores in its GPU chips. To facilitate writing programs that involve a general-purpose processor and a GPU, an extension to the C programming language, called CUDA C, has been developed by NVIDIA [1, 2]. This extension enables a single program to be written in C, with special keywords used to label the functions executed by the processing cores in a GPU chip. The compiler and related software tools automatically partition the final object program into the portions that are translated into machine instructions for the host computer and the GPU chip. Library routines are provided to allocate storage in the dedicated memory of a GPU-based video card and to transfer data between the main memory and the dedicated memory. An open standard called OpenCL has also been proposed by industry as a programming framework for systems that include GPU chips from any vendor [3].

12.3 SHARED-MEMORY MULTIPROCESSORS

A multiprocessor system consists of a number of processors capable of simultaneously executing independent tasks. The granularity of these tasks can vary considerably. A task may encompass a few instructions for one pass through a loop, or thousands of instructions executed in a subroutine.

In a shared-memory multiprocessor, all processors have access to the same memory. Tasks running in different processors can access shared variables in the memory using the same addresses. The size of the shared memory is likely to be large. Implementing a large memory in a single module would create a bottleneck when many processors make requests to access the memory simultaneously. This problem is alleviated by distributing the memory

across multiple modules so that simultaneous requests from different processors are more likely to access different memory modules, depending on the addresses of those requests.

An interconnection network enables any processor to access any module that is a part of the shared memory. When memory modules are kept physically separate from the processors, all requests to access memory must pass through the network, which introduces latency. Figure 12.2 shows such an arrangement. A system which has the same network latency for all accesses from the processors to the memory modules is called a Uniform Memory Access (UMA) multiprocessor. Although the latency is uniform, it may be large for a network that connects many processors and memory modules.

For better performance, it is desirable to place a memory module close to each processor. The result is a collection of *nodes*, each consisting of a processor and a memory module. The nodes are then connected to the network, as shown in Figure 12.3. The network latency is avoided when a processor makes a request to access its local memory. However, a request to access a remote memory module must pass through the network. Because of the difference in latencies for accessing local and remote portions of the shared memory, systems of this type are called Non-Uniform Memory Access (NUMA) multiprocessors.

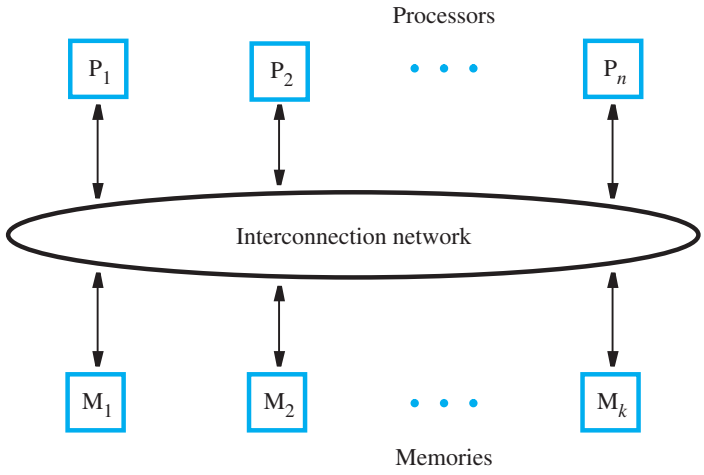


Figure 12.2 A UMA multiprocessor.

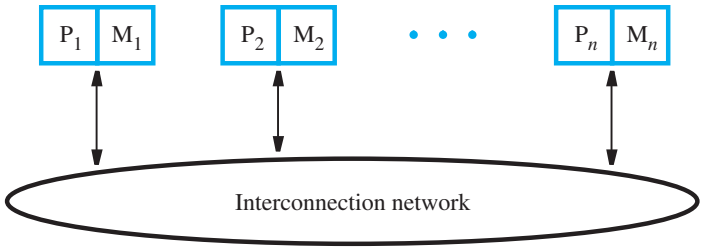


Figure 12.3 A NUMA multiprocessor.

12.3.1 INTERCONNECTION NETWORKS

The interconnection network must allow information transfer between any pair of nodes in the system. The network may also be used to broadcast information from one node to many other nodes. The traffic in the network consists of requests (such as read and write) and data transfers.

The suitability of a particular network is judged in terms of cost, bandwidth, effective throughput, and ease of implementation. The term *bandwidth* refers to the capacity of a transmission link to transfer data and is expressed in bits or bytes per second. The *effective throughput* is the actual rate of data transfer. This rate is less than the available bandwidth because a given link must also carry control information that coordinates the transfer of data.

Information transfer through the network usually takes place in the form of *packets* of fixed length and specified format. For example, a read request is likely to be a single packet sent from a processor to a memory module. The packet contains the node identifiers for the source and destination, the address of the location to be read, and a command field that indicates what type of read operation is required. A write request that writes one word in a memory module is also likely to be a single packet that includes the data to be written. On the other hand, a read response may involve an entire cache block requiring several packets for the data transfer.

Ideally, a complete packet would be handled in parallel in one clock cycle at any node or switch in the network. This implies having wide links, comprising many wires. However, to reduce cost and complexity, the links are often considerably narrower. In such cases, a packet must be divided into smaller pieces, each of which can be transmitted in one clock cycle.

The following sections describe a few of the interconnection networks that are commonly used in multiprocessors.

Bus

A *bus* is a set of lines (wires) that provide a single shared path for information transfer, as discussed in Chapter 7. Buses are most commonly used in UMA multiprocessors to connect a number of processors to several shared-memory modules. Arbitration is necessary to ensure that only one of many possible requesters is granted use of the bus at any time. The bus is suitable for a relatively small number of processors because of the contention for access to the bus and the increased propagation delays caused by electrical loading when many processors are connected.

A simple bus does not allow a new request to appear on the bus until the response for the current request has been provided. However, if the response latency is high, there may be considerable idle time on the bus.

Higher performance can be achieved by using a *split-transaction* bus, in which a request and its corresponding response are treated as separate events. Other transfers may take place between them. Consider a situation where multiple processors need to make read requests to the memory. Arbitration is used to select the first processor to be granted use of the bus for its request. After the request is made, a second processor is selected to make its request, instead of leaving the bus idle. Assuming that this request is to a different memory module, the two read accesses proceed in parallel. If neither module has finished with its access, a third processor is selected to make its request, and so on. Eventually, one memory

module completes its read access. It is granted the use of the bus to transfer the data to the requesting processor. As other modules complete their accesses, the bus is used to transfer their responses. The actual length of time between each request and its corresponding response may vary as requests and responses for different transactions with the memory are interleaved on the bus to make efficient use of the available bandwidth.

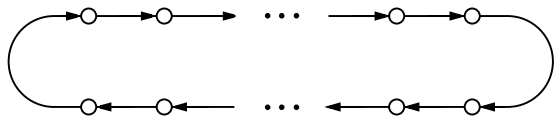
The split-transaction bus requires a more complex bus protocol. The main source of complexity is the need to match each response with its corresponding request. This is usually handled by associating a unique tag with each request that appears on the bus. Each response then appears with the appropriate tag so that the source can match it to its original request.

Ring

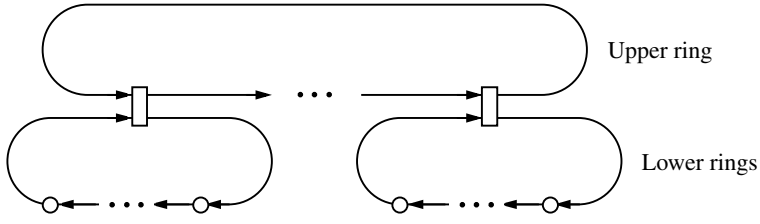
A *ring* network is formed with point-to-point connections between nodes, as shown in Figure 12.4. A single ring is shown in Figure 12.4*a*. A long single ring results in high average latency for communication between any two nodes. This high latency can be mitigated in two different ways.

A second ring can be added to connect the nodes in the opposite direction. The resulting *bidirectional ring* halves the average latency and doubles the bandwidth. However, handling of communications is more complex.

Another approach is to use a *hierarchy* of rings. A two-level hierarchy is shown in Figure 12.4*b*. The upper-level ring connects the lower-level rings. The average latency for communication between any two nodes on lower-level rings is reduced with this arrangement. Transfers between nodes on the same lower-level ring need not traverse the upper-level ring. Transfers between nodes on different lower-level rings include a traversal on part of the upper-level ring. The drawback of the hierarchical scheme is that the upper-level ring may become a bottleneck when many nodes on different lower-level rings communicate with each other frequently.



(a) Single ring



(b) Hierarchy of rings

Figure 12.4 Ring-based interconnection networks.

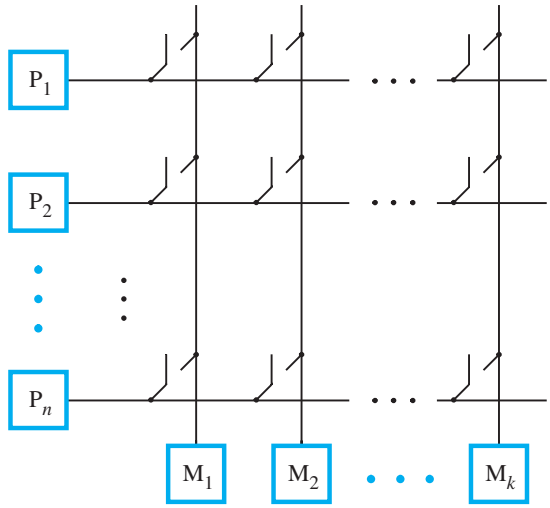


Figure 12.5 Crossbar interconnection network.

Crossbar

A *crossbar* is a network that provides a direct link between any pair of units connected to the network. It is typically used in UMA multiprocessors to connect processors to memory modules. It enables many simultaneous transfers if the same destination is not the target of multiple requests. For example, we can implement the structure in Figure 12.2 using a crossbar that comprises a collection of switches as in Figure 12.5. For n processors and k memories, $n \times k$ switches are needed.

Mesh

A natural way of connecting a large number of nodes is with a two-dimensional *mesh*, as shown in Figure 12.6. Each internal node of the mesh has four connections, one to each of its horizontal and vertical neighbors. Nodes on the boundaries and corners of the mesh

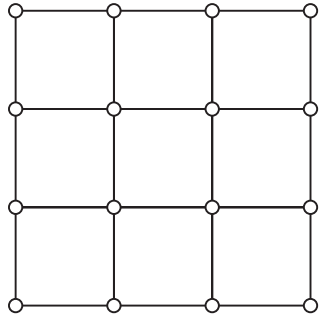


Figure 12.6 A two-dimensional mesh network.

have fewer neighbors and hence fewer connections. To reduce latency for communication between nodes that would otherwise be far apart in the mesh, wraparound connections may be introduced between nodes at opposite boundaries of the mesh. A network with such connections is called a *torus*. All nodes in a torus have four connections. Average latency is reduced, but the implementation complexity for routing requests and responses through a torus is somewhat higher than in the case of a simple mesh.

12.4 CACHE COHERENCE

A shared-memory multiprocessor is easy to program. Each variable in a program has a unique address location in the memory, which can be accessed by any processor. However, each processor has its own cache. Therefore, it is necessary to deal with the possibility that copies of shared data may reside in several caches. When any processor writes to a shared variable in its own cache, all other caches that contain a copy of that variable will then have the old, incorrect value. They must be informed of the change so that they can either update their copy to the new value or invalidate it. This is the issue of maintaining *cache coherence*, which requires having a consistent view of shared data in multiple caches.

In Chapter 8 we discussed two basic approaches for performing write operations on data in a cache. The write-through approach changes the data in both the cache and the main memory. The write-back approach changes the data only in the cache; the main memory copy is updated when a modified data block in the cache has to be replaced. Similar approaches can be used to address cache coherence in a multiprocessor system.

12.4.1 WRITE-THROUGH PROTOCOL

A write-through protocol can be implemented in one of two ways. One version is based on updating the values in other caches, while the second relies on invalidating the copies in other caches.

Let us consider the *update* protocol first. When a processor writes a new value to a block of data in its cache, the new value is also written into the memory module containing the block being modified. Since copies of this block may exist in other caches, these copies must be updated to reflect the change caused by the Write operation. The simplest way of doing this is to broadcast the written data to the caches of all processors in the system. As each processor receives the broadcast data, it updates the contents of the affected cache block if this block is present in its cache.

The second version of the write-through protocol is based on *invalidation* of copies. When a processor writes a new value into its cache, this value is also sent to the appropriate location in memory, and all copies in other caches are invalidated. Again, broadcasting can be used to send the invalidation requests throughout the system.

12.4.2 WRITE-BACK PROTOCOL

Maintaining coherence with the write-back protocol is based on the concept of ownership of a block of data in the memory. Initially, the memory is the owner of all blocks, and the memory retains ownership of any block that is read by a processor to place a copy in its cache.

If some processor wants to write to a block in its cache, it must first become the exclusive owner of this block. To do so, all copies in other caches must first be invalidated with a broadcast request. The new owner of the block may then modify the contents at will without having to take any other action.

When another processor wishes to read a block that has been modified, the request for the block must be forwarded to the current owner. The data are then sent to the requesting processor by the current owner. The data are also sent to the appropriate memory module, which reacquires ownership and updates the contents of the block in the memory. The cache of the processor that was the previous owner retains a copy of the block. Hence, the block is now shared with copies in two caches and the memory. Subsequent requests from other processors to read the same block are serviced by the memory module containing the block.

When another processor wishes to write to a block that has been modified, the current owner sends the data to the requesting processor. It also transfers ownership of the block to the requesting processor and invalidates its cached copy. Since the block is being modified by the new owner, the contents of the block in the memory are not updated. The next request for the same block is serviced by the new owner.

The write-back protocol has the advantage of creating less traffic than the write-through protocol. This is because a processor is likely to perform several writes to a cache block before this block is needed by another processor. With the write-back protocol, these writes are performed only in the cache, once ownership is acquired with an invalidation request. With the write-through protocol, each write must also be performed in the appropriate memory module and broadcast to other caches.

So far, we have assumed that update and invalidate requests in these protocols are broadcast through the interconnection network. Whether it is easy to implement such broadcasts depends largely on the structure of the interconnection network. The most natural network for supporting broadcasting is the single bus. In multiprocessors that connect a modest number of processors to the memory modules using a single bus, cache coherence can be realized using a scheme known as snooping.

12.4.3 SNOOPY CACHES

In a single-bus system, all transactions between processors and memory modules occur via requests and responses on the bus. In effect, they are broadcast to all units connected to the bus. Suppose that each processor cache has a controller circuit that observes, or *snoops*, all transactions on the bus. We now describe some scenarios for the write-back protocol and how cache coherence is enforced.

Consider a processor that has previously read a copy of a block from the memory into its cache. Before writing to this block for the first time, the processor must broadcast an *invalidation request* to all other caches, whose controllers accept the request and invalidate any copies of the same block. This action causes the requesting processor to become the new owner of the block. The processor may then write to the block and mark it as being modified. No further broadcasts are needed from the same processor to write to the modified block in its cache.

Now, if another processor broadcasts a *read request* on the bus for the same block, the memory must not respond because it is not the current owner of the block. The processor owning the requested block snoops the read request on the bus. Because it holds a modified copy of the requested block in its cache, it asserts a special signal on the bus to prevent the memory from responding. The owner then broadcasts a copy of the block on the bus, and marks its copy as clean (unmodified). The data response on the bus is accepted by the cache of the processor that issued the read request. The data response is also accepted by the memory to update its copy of the block. In this case, the memory reacquires ownership of the block, and the block is said to be in a shared state because copies of it are in the caches of two processors. Coherence is maintained because the two cached copies and the copy of the block in the memory contain the same data. Subsequent requests from any processor are serviced by the memory.

Consider now the situation in which two processors have copies of the same block in their respective caches, and both processors attempt to write to the same cache block at the same time. Since the block is in the shared state, the memory is the owner of the block. Hence, both processors request the use of the bus to broadcast an invalidation message. One of the processors is granted the use of the bus first. That processor broadcasts its invalidation request and becomes the new owner of the block. Through snooping, the copy of the block in the cache of the other processor is invalidated. When the other processor is later granted the use of the bus, it broadcasts a *read-exclusive request*. This request combines a read request and an invalidation request for the same block. The controller for the first processor snoops the read-exclusive request, provides a data response on the bus, and invalidates the copy in its cache. Ownership of the block is therefore transferred to the second processor making the request. The memory is not updated because the block is being modified again. Since the requests from the two processors are handled sequentially, cache coherence is maintained at all times.

The scheme just described is based on the ability of cache controllers to observe the activity on the bus and take appropriate actions. Such schemes are called *snoopy-cache* techniques.

For performance reasons, it is important that the snooping function not interfere with the normal operation of a processor and its cache. Such interference occurs if the cache controller accesses the tags of the cache for every request that appears on the bus. In most cases, the cache would not contain a valid copy of the block that is relevant to a request. To eliminate unnecessary interference, each cache can be provided with a set of duplicate tags, which maintain the same status information about the blocks in the cache but can be accessed separately by the snooping circuitry.

12.4.4 DIRECTORY-BASED CACHE COHERENCE

The concept of snoopy caches is easy to implement in single-bus systems. Large shared-memory multiprocessors use interconnection networks such as rings and meshes. In such systems, broadcasting every single request to the caches of all processors is inefficient. A scalable, but more complex, solution to this problem uses *directories* in each memory module to indicate which nodes may have copies of a given block in the shared state. If a block is modified, the directory identifies the node that is the current owner. Each request from a processor must be sent first to the memory module containing the relevant block. The directory information for that block is used to determine the action that is taken. A read request is forwarded to the current owner if the block is modified. In the case of a write request for a block that is shared, individual invalidations are sent only to nodes that may have copies of the block in question. The cost and complexity of the directory-based approach for enforcing cache coherence limits its use to large systems. Small multiprocessors, including current multicore chips, typically use snooping.

12.5 MESSAGE-PASSING MULTICOMPUTERS

A different way of using multiple processors involves implementing each node in the system as a complete computer with its own memory. Other computers in the system do not have direct access to this memory. Data that need to be shared are exchanged by sending messages from one computer to another. Such systems are called *message-passing multicomputers*.

Parallel programs are written differently for message-passing multicomputers than for shared-memory multiprocessors. To share data between nodes, the program running in the computer that is the source of the data must send a message containing the data to the destination computer. The program running in the destination computer receives the message and copies the data into the memory of that node.

To facilitate message passing, a special communications unit at each node is often responsible for the low-level details of formatting and interpreting messages that are sent and received, and for copying message data to and from the memory of the node. The computer in each node issues commands to the communications unit. The computer then continues performing other computations while the communications unit handles the details of sending and receiving messages.

12.6 PARALLEL PROGRAMMING FOR MULTIPROCESSORS

The preceding sections described hardware arrangements for shared-memory multiprocessors that can exploit parallelism in application programs. The available parallelism may be found in loops with independent passes, and also in independent higher-level tasks.

A source program written in a high-level language allows a programmer to express the desired computation in a manner that is easy to understand. It must be translated by

the compiler and the assembler into machine-language representation. The hardware of the processor is designed to execute machine-language instructions in proper sequence to perform the computation desired by the programmer. It cannot automatically identify independent high-level tasks that could be executed in parallel. The compiler also has limitations in detecting and exploiting parallelism. It is therefore the responsibility of the programmer to explicitly partition the overall computation in the source program into tasks and to specify how they are to be executed on multiple processors.

Programming for a shared-memory multiprocessor is a natural extension of conventional programming for a single processor. A high-level source program is written using tasks that are executed by one processor. But it is also possible to indicate that certain tasks are to be executed simultaneously in different processors. Sharing of data is achieved by defining global variables that are read and written by different processors as they perform their assigned tasks. The multicore chips currently used in general-purpose computers, such as those implementing the Intel IA-32 architecture, are programmed in this manner.

To illustrate parallel programming, we consider the example of computing the dot product of two vectors, each containing N numbers. A C-language program for this task is shown in Figure 12.7. The details of initializing the contents of the two vectors are omitted to focus on the aspects relevant to parallel programming.

The loop accumulates the sum of N products. Each pass depends on the partial sum computed in the preceding pass, and the result computed in the final pass is the dot product. Despite the dependency, it is possible to partition the program into independent tasks for simultaneous execution by exploiting the associative property of addition. Each task computes a partial sum, and the final result is obtained by adding the partial sums.

```
#include    < stdio.h>      /* Routines for input/output. */

#define     N    100        /* Number of elements in each vector. */

double      a[N], b[N];     /* Vectors for computing the dot product. */

void        main (void)
{
            int i;
            double dot_product;

            < Initialize vectors a[ ], b[ ] – details omitted.>
            dot_product = 0.0;
            for (i = 0; i <  N; i++)
               dot_product = dot_product + a[i] * b[i];
            printf ("The dot product is %g\ n", dot_product);
}
```

Figure 12.7 C program for computing a dot product.

To implement a parallel program for computing the dot product, two questions need to be answered:

- How do we make multiple processors participate in parallel execution to compute the partial sums?
- How do we ensure that each processor has computed its partial sum before the final result for the dot product is computed?

Thread Creation

To answer the first question, we define the tasks that are assigned to different processors, and then we describe how execution of these tasks is initiated in multiple processors. We can write a parallel version of the dot product program using parameters for the number of processors, P, and the number of elements in each vector, N. We assume for simplicity that N is evenly divisible by P. The overall computation involves a sum of N products. For P processors, we define P independent tasks, where each task is the computation of a partial sum of N/P products.

When a program is executed in a single processor, there is one active thread of execution control. This thread is created implicitly by the operating system (OS) when execution of the program begins. For a parallel program, we require the independent tasks to be handled separately by multiple threads of execution control, one for each processor. These threads must be created explicitly. A typical approach is to use a routine named *create_thread* in a library that supports parallel programming. The library routine accepts an input parameter, which is a pointer to a subroutine to be executed by the newly created thread. An operating system service is invoked by the library routine to create a new thread with a distinct stack, so that it may call other subroutines and have its own local variables. All global variables are shared among all threads.

It is necessary to distinguish the threads from each other. One approach is to provide another library routine called *get_my_thread_id* that returns a unique integer between 0 and $P - 1$ for each thread. With that information, a thread can determine the appropriate subset of the overall computation for which it is responsible.

Thread Synchronization

The second question involves determining when threads have completed their tasks, so that the final result can be computed correctly. *Synchronization* of multiple threads is therefore required. There are several methods of synchronization, and they are often implemented in additional library routines for parallel programming. Here, we consider one method called a *barrier*.

The purpose of a barrier is to force threads to wait until they have all reached a specific point in the program where a call is made to the library routine for the barrier. Each thread that calls the barrier routine enters a busy-wait loop until the last thread calls the routine and enables all of the threads to continue their execution. This ensures that the threads have completed their respective computations preceding the barrier call.

Example Parallel Program

Having described the issues related to thread creation and synchronization, and typical library routines that are provided for thread management, we can now present a parallel dot product program as an example. Figure 12.8 shows a *main* routine, and another routine

```
#include    < stdio.h>          /* Routines for input/output. */
#include    "threads.h"         /* Routines for thread creation/synchronization. */

#define     N    100            /* Number of elements in each vector. */
#define     P    4              /* Number of processors for parallel execution. */

double      a[N], b[N];         /* Vectors for computing the dot product. */
double      partial_sums[P];    /* Array of results computed by threads. */
Barrier     bar;                /* Shared variable to support barrier synchronization. */

void        ParallelFunction (void)
{
            int my_id, i, start, end;
            double s;

            my_id = get_my_thread_id ();     /* Get unique identifier for this thread. */
            start = (N/P) * my_id;           /* Determine start/end using thread identifier. */
            end = (N/P) * (my_id + 1) – 1;   /* N is assumed to be evenly divisible by P .* /
            s = 0.0;
            for (i = start; i <= end; i++)
               s = s + a[i] * b[i];
            partial_sums[my_id] = s;         /* Save result in array. */
            barrier (&bar, P);               /* Synchronize with other threads. */
}

void        main (void)
{
            int i;
            double dot_product;

            < Initialize vectors a[], b[] – details omitted.>
            init_barrier (&bar);
            for (i = 1; i <  P; i++)         /* Create P – 1 additional threads. */
               create_thread (ParallelFunction);
            ParallelFunction();              /* Main thread also joins parallel execution. */
            dot_product = 0.0;               /* After barrier synchronization, compute final result. */
            for (i = 0; i <  P; i++)
               dot_product = dot_product + partial_sums[i];
            printf ("The dot product is %g\ n", dot_product);
}
```

Figure 12.8 Parallel program in C for computing a dot product.

called *ParallelFunction* that defines the independent tasks for parallel execution. When the program begins executing, there is only one thread executing the *main* routine. This thread initializes the vectors, then it initializes a shared variable needed for barrier synchronization. To initiate parallel execution, the *create_thread* routine is called $P - 1$ times from the *main* routine to create additional threads that each execute *ParallelFunction*. Then, the thread executing the *main* routine calls *ParallelFunction* directly so that a total of P threads are involved in the overall computation. The operating system software is responsible for distributing the threads to different processors for parallel execution.

Each thread calls *get_my_thread_id* from *ParallelFunction* to obtain a unique integer identifier in the range 0 to $P - 1$. Using this information, the thread calculates the *start* and *end* indices for the loop that generates the partial sum of that thread. After executing the loop, it writes the result to a separate element of the shared *partial_sums* array using its unique identifier as the array index. Then, the thread calls the library routine for barrier synchronization to wait for other threads to complete their computation.

After the last thread to complete its computation calls the barrier routine, all threads return to *ParallelFunction*. There is no further computation to perform in *ParallelFunction*, so the $P - 1$ threads created by the library call in the *main* routine terminate. The thread that called *ParallelFunction* directly from the *main* routine returns to compute the final result using the values in the *partial_sums* array.

The program in Figure 12.8 uses generic library routines to illustrate thread creation and synchronization. A large collection of routines for parallel programming in the C language is defined in the IEEE 1003.1 standard [4]. This collection is also known as the *POSIX threads* or *Pthreads* library. It provides a variety of thread management and synchronization mechanisms. Implementations of this library are available for widely-used operating systems to facilitate programming for multiprocessors.

12.7 PERFORMANCE MODELING

The most important measure of the performance of a computer is how quickly it can execute programs. When considering one processor, the speed with which instructions are fetched and executed is affected by the instruction set architecture and the hardware design. The total number of instructions that are executed is affected by the compiler as well as the instruction set architecture. Chapter 6 introduced the basic performance equation, which is a low-level mathematical model that reflects these considerations for one processor. The terms in that model include the number of instructions executed, the average number of cycles per instruction, and the clock frequency. This model enables the prediction of execution time, given sufficiently detailed information.

A higher-level model that relies on less detailed information can be used to assess potential improvements in performance. Consider a program whose execution time on some computer is T_{orig}. Our objective is to assess the extent to which the execution time can be reduced when a performance enhancement, such as parallel processing, is introduced.

Assume that a fraction f_{enh} of the execution time is affected by the enhancement. The remaining fraction, $f_{unenh} = 1 - f_{enh}$, is unchanged. Let p represent the factor by which the portion of time $f_{enh} \times T_{orig}$ is reduced due to the performance enhancement. The new execution time is

$$T_{new} = T_{orig} \, (f_{unenh} + f_{enh}/p)$$

The *speedup* is the ratio T_{orig}/T_{new} or

$$1/\,(f_{unenh} + f_{enh}/p)$$

The above expression for speedup is known as *Amdahl's Law*. It is named after Gene Amdahl, who was the first to formalize the reasoning explained above. It is a way of stating the intuitive observation that the benefit of a given performance enhancement increases if it affects a larger portion of the execution time.

Once a breakdown of the original execution time has been determined, it is often useful to determine an upper bound on the possible speedup. To do so, we let $p \rightarrow \infty$ to reflect the ideal, but unrealistic, reduction of the fraction f_{enh} of execution time to zero. The resulting speedup is $1/f_{unenh}$, which means that the portion of execution time that is not enhanced is the limiting factor on performance. A smaller value of f_{unenh} gives a larger bound on the speedup. For example, $f_{unenh} = 0.1$ gives an upper bound of 10, but $f_{unenh} = 0.05$ gives a larger bound of 20. However, the expected speedup using a realistic value of p is normally well below the upper bound. For example, using $p = 16$ with $f_{unenh} = 0.05$ gives a speedup of only $1/(0.05 + 0.95/16) = 9.1$, well below the upper bound of 20.

The important conclusion from this discussion is that the unenhanced portion of the original execution time can significantly limit the achievable speedup, even if the enhanced portion is improved by an arbitrarily large factor. If a programmer can determine, even approximately, the fractions f_{unenh} and f_{enh} of execution time before applying a particular enhancement, Amdahl's Law can provide useful insight into the expected improvement. That information can be used to determine whether the expected gain in performance justifies the effort and expense to implement the enhancement.

12.8 CONCLUDING REMARKS

Fundamental techniques such as pipelining and caches are important ways of improving performance and are widely used in computers. The additional techniques of multithreading, vector (SIMD) processing, and multiprocessing provide the potential for further improvements in performance by making more efficient use of processing resources and by performing more operations in parallel. These techniques have been incorporated into general-purpose multicore processor chips.

PROBLEMS

12.1 **[M]** Assume that a bus transfer takes T seconds and memory access time is $4T$ seconds. A read request over a conventional bus then requires $6T$ seconds to complete. How many conventional buses are needed to equal or exceed the effective throughput of a split-transaction bus that operates with the same time delays? Consider only read requests, ignore memory conflicts, and assume that all memory modules are connected to all buses in the multiple-bus case. Does your answer increase or decrease if memory access time increases?

12.2 **[M]** Assume that a computation comprises $k + 1$ distinct tasks. In order to prepare a program for the desired computation, each of these tasks has been written as a function in the C language. The $k + 1$ functions are labeled T0(), T1(), . . . , Tk(). Each function requires τ time units to execute. Due to data dependencies, functions T1() to Tk() must be executed after function T0(). There are no data dependencies among the functions T1() to Tk().

(a) Using the given functions, write a C program that executes on a single processor.

(b) Write an equivalent C program that executes on k processors.

(c) Derive an expression for the ideal speedup for the program in part (b) relative to the program in part (a).

12.3 **[D]** What are the arguments for and against invalidation and updating as strategies for maintaining cache coherence?

12.4 **[D]** The approaches of shared memory and message passing both support simultaneous execution of tasks that interact with each other. Which of these two approaches can emulate the action of the other more easily? Briefly justify your answer.

12.5 **[E]** With reference to Figure 12.1, assume an array size of $N = 32$. Let all instructions require one time unit to execute in a processor. Determine the speedup of vectorization for vector lengths of 4, 8, 16, and 32.

12.6 **[M]** For vectorization, efficiency is defined as the ratio of the speedup to the vector length. Determine the efficiency for each of the results in Problem 12.5. Comment on the variation of the efficiency with the vector length.

12.7 **[M]** In the parallel program in Figure 12.8 for computing the dot product, the sum of the partial products is computed by one processor after all threads have reached the barrier. Modify the program so that each thread adds its partial sum incrementally to a global sum for the dot product. To prevent two or more threads from trying to update the global sum at the same time, synchronization is required. One method is to use a shared counter variable whose value is the identifier of the thread that is granted exclusive access to the variable for the global sum. After the thread with exclusive access has updated the global sum, it can simply increment this counter to grant exclusive access to another thread.

12.8 **[E]** Assume that a multiprocessor has eight processors. Based on an existing program that runs on one processor, a parallel program is written to run on this multiprocessor. Assume that the workload of the parallel portion of the program can be distributed evenly over the

eight processors. Use Amdahl's Law to determine the value of f_{enh} that would allow a speedup of 5.

REFERENCES

1. NVIDIA Corporation, *NVIDIA C CUDA Programming Guide*, Version 3.1.1, July 2010, available at http://www.nvidia.com.
2. D. Kirk and W.-M. Hwu, *Programming Massively Parallel Processors: A Hands-on Approach*, Morgan Kaufmann, Burlington, Massachusetts, 2010.
3. Khronos Group, *OpenCL Introduction and Overview*, June 2010, available at http://www.khronos.org/opencl.
4. IEEE, *1003.1 Standard for Information Technology—Portable Operating System Interface (POSIX): System Interfaces, Issue 6*, 2001, available at http://www.ieee.org.